

OPEN UNIVERSITY OF SRI LANKA

Faculty of Natural Science

CODE SUBMISSION

MINI PROJECT
(Group no:03)

OUSL STUDENT MAP

Submitted by:
H M Tharaka Sadharuwan
Group leader
Student ID: 124682005

Other Group members:

1. **O K Kalana Indusara : 724682892**
2. **A G D R Jayaweera : 624698181**
3. **G W G C B Sanjeewa : 524655985**
4. **I V C M Bandara : 724655849**
5. **K L K P Liyanage : 524679572**
6. **R S Warnakulasooriya : 624683193**

Course code: COU4303
Course name: Artificial Intelligence
Academic Year : 2024/2025

PROLOG CODE

```
/*====== OUSL STUDENT MAP=====*/

/*
Start : Embilipitiya (Student live location)
Destination: Open University of Sri Lanka (Nawala)
*/

/*Format~
road(From, To, Transport method , Time(minutes), Cost(Rs)).
*/

/*==== ROUTE 01 ~Rathnapura road normal bus ====*/

road(embilipitiya, maradana, normal_bus, 300, 650).
road(maradana, narahenpita, normal_bus, 30, 50).

/* Walking */

road(narahenpita, open_university, walk, 25, 0).

/* Taxi */

road(narahenpita, open_university, taxi, 10, 200).



/*==== ROUTE 02~ Higyway ====*/

road(embilipitiya, maharagama, highway_bus, 150, 1600).
road(maharagama, nugegoda, normal_bus, 25, 60).
road(nugegoda, open_university, normal_bus, 15, 50).

/*==== ROUTE 03~ Train ====*/

road(embilipitiya, matara, normal_bus, 100, 250).
road(matara,maradana, train, 210, 300).

/*==== ROAD IS AVAILABLE ====*/
blocked(_, _) :- fail.

available_road(A, B, Transport, Time, Cost) :-
    road(A, B, Transport, Time, Cost),
    \+ blocked(A, B),
    \+ blocked(B, A).
```

```
/* Roads are considered both directions */
connected(A, B, Transport, Time, Cost) :-
   available_road(A, B, Transport, Time, Cost).

connected(A, B, Transport, Time, Cost) :-
   available_road(B, A, Transport, Time, Cost).

/*==== ROUTE DEFINITION ====*/


route(1,
     'Ratnapura Road - Normal Bus',
     [embilipitiya, maradana, narahenpita, open_university]).

route(2,
     'Highway Bus',
     [embilipitiya, maharagama, nugegoda, open_university]).

route(3,
     'Bus + Train',
     [embilipitiya, matara, maradana,
      narahenpita, open_university]).

/*==== CALCULATE TIME ====*/

time([_], 0).

time([A,B|Rest], Time):-
          available_road(A, B, _, T, _),
          time([B|Rest], Remaining),
          Time is T + Remaining.

/*==== CALCULATE COST ====*/

cost([_],0).

cost([A,B|Rest], Cost):-
          available_road(A, B, _, _, C),
          cost([B|Rest], Remaining),
          Cost is C + Remaining.

/*==== DISPLAY DETAILS ====*/

display_details([_]):-
          write(' == Total == '),nl,nl.
```

```
display_details([A, B|Rest]):-
        connected(A, B, Transport, Time, Cost),
        write(A), write('--->'), write(B),nl,
        write('     Transport method: '),write(Transport),nl,
        write('     Time: '),write(Time),write(' minutes'),nl,
        write('     Cost:Rs '),write(Cost),nl,nl,
        display_details([B|Rest]).


/*==== DISPLAY ONE ROUTE ====*/

display_route(Number):-

        route(Number, Name, Path),

        write('Route '), write(Number),write(':  '),write(Name),nl,nl,

        display_details(Path),
        time(Path, Time),
        cost(Path, Cost),

        write('Total Time: '),write(Time),write(' minutes'),nl,
        write('Total Cost: Rs. '),write(Cost),nl.


/*==== DISPLAY ALL ROUTES ====*/   /*Note accessory */

display_all_routes:-
        write('====OUSL STUDENT MAP===='),nl,nl,
        write('Starting Location: Embilipitiya'),nl,
        write('Destination: Open University of Sri Lanka(Nawala)'),nl,nl,

         ( display_route(1) -> true ;
        writeln('Route 1 is Blocked.')),nl,
        ( display_route(2) -> true ;
        writeln('Route 2 is Blocked.')),nl,
        ( display_route(3) -> true ;
        writeln('Route 3 is Blocked.')).



/*==== SELECT A ROUTE ====*/
start:-

select_route.
```

```
select_route:-
          display_all_routes,

          write('Select available route (1-3): '),
          read(Choice),
          (Choice = 1 -> selected_route(1);
          Choice = 2 -> selected_route(2);
          Choice = 3 -> selected_route(3);
          writeln('Invalid choice! Please enter 1, 2 or 3.'),
      select_route
   ).
/*==== DISPLAY SELECTED ROUTE ====*/
selected_route(Number):-
          route(Number, Name, Path),

          write('=====SELECTED ROUTE====='),nl,
          write(Number),nl,nl,
          write('Route: '),write(Name),nl,nl,
          write('Journey: '),nl,
          display_details(Path),nl,
          time(Path, Time),
          cost(Path, Cost),nl,
          write('Total Time: '),write(Time),write(' minutes'),nl,
          write('Total Cost:Rs.'),write(Cost).



/*
  12. BREADTH FIRST SEARCH (BFS)
*/

bfs(Start, Goal, Path) :-
   bfs_queue([[Start]], Goal, ReversePath),
   reverse(ReversePath, Path).


bfs_queue([[Goal|Rest]|_], Goal, [Goal|Rest]).

bfs_queue([[Current|Rest]|Queue], Goal, Path) :-
   findall(
      [Next,Current|Rest],
      (
         connected(Current, Next, _, _, _),
         \+ member(Next, [Current|Rest])
      ),
      NewPaths
   ),
```

```
    append(Queue, NewPaths, NewQueue),

    bfs_queue(NewQueue, Goal, Path).


/*
  13. DEPTH FIRST SEARCH (DFS)
*/

dfs(Start, Goal, Path) :-
    dfs_search(Start, Goal, [Start], ReversePath),
    reverse(ReversePath, Path).


dfs_search(Goal, Goal, Visited, Visited).

dfs_search(Current, Goal, Visited, Path) :-
    connected(Current, Next, _, _, _),
    \+ member(Next, Visited),
    dfs_search(Next, Goal, [Next|Visited], Path).


/*
  14. A* SEARCH
  f(n) = g(n) + h(n)
*/
/* Heuristic values
  Estimated remaining travel time to Open University
*/
heuristic(embilipitiya, 340).
heuristic(maradana, 40).
heuristic(narahenpita, 25).
heuristic(maharagama, 40).
heuristic(matara, 250).
heuristic(open_university, 0).
/* A* main predicate */
astar(Start, Goal, Path, Cost) :-
    heuristic(Start, H),
    astar_search(
        [node(Start, 0, H, [Start])],
        Goal,
        Path,
        Cost
    ).
/* Goal reached */
```

```
astar_search(
   [node(Goal, Cost, _, ReversePath)|_],
   Goal,
   Path,
   Cost
) :-
   reverse(ReversePath, Path).
/* Expand lowest f-cost node */

astar_search(
   [node(Current, G, _, ReversePath)|Open],
   Goal,
   Path,
   Cost
) :-

   findall(
      node(
         Next,
         NewG,
         F,
         [Next|ReversePath]
      ),
      (
         connected(Current, Next, _, StepCost, _),
         \+ member(Next, ReversePath),

         NewG is G + StepCost,

         heuristic(Next, H),

         F is NewG + H
      ),
      Children
   ),

   append(Open, Children, NewOpen),

   sort_open_list(NewOpen, SortedOpen),

   astar_search(SortedOpen, Goal, Path, Cost).


/* Sort nodes according to F value */

sort_open_list(Open, Sorted) :-
   predsort(compare_nodes, Open, Sorted).
```

```
compare_nodes(Order,
            node(_,_,F1,_),
            node(_,_,F2,_)) :-
   compare(Order, F1, F2).

/*
  15. COMPARE BFS, DFS AND A*
*/

compare_algorithms :-
   write('========================================'), nl,
   write('       ALGORITHM COMPARISON'), nl,
   write('========================================'), nl,

   /* BFS */
   bfs(embilipitiya, open_university, BFSPath),
   time(BFSPath, BFSTime),
   cost(BFSPath, BFSCost),

   write('BFS'), nl,
   write('Path : '), write(BFSPath), nl,
   write('Time : '), write(BFSTime), write(' minutes'), nl,
   write('Cost : Rs.'), write(BFSCost), nl,

   /* DFS */
   dfs(embilipitiya, open_university, DFSPath),
   time(DFSPath, DFSTime),
   cost(DFSPath, DFSCost),

   write('DFS'), nl,
   write('Path : '), write(DFSPath), nl,
   write('Time : '), write(DFSTime), write(' minutes'), nl,
   write('Cost : Rs.'), write(DFSCost), nl,

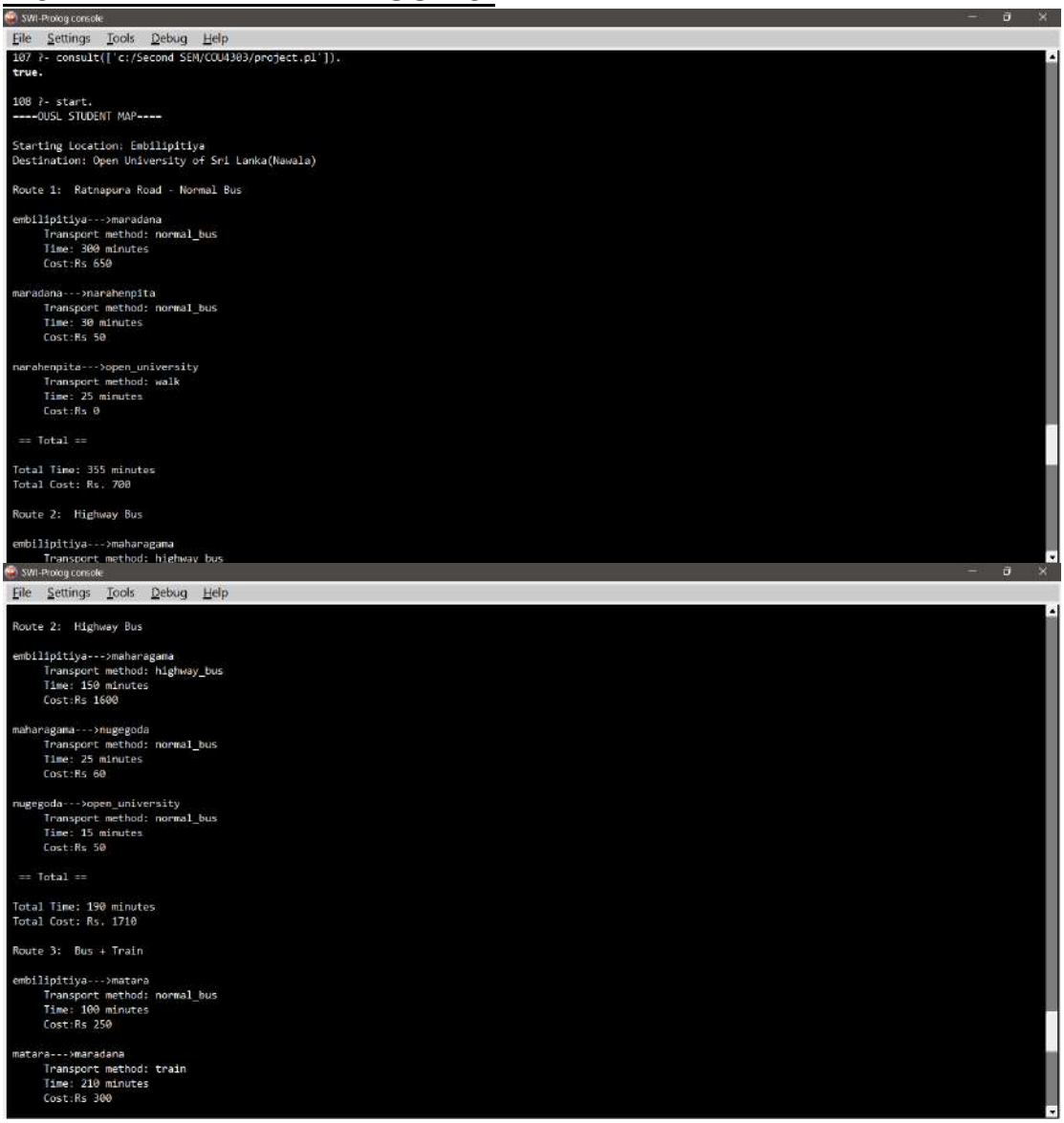
   /* A* */
   astar(embilipitiya, open_university, AStarPath, AStarCost),
   time(AStarPath, AStarTime),

   write('A*'), nl,
   write('Path : '), write(AStarPath), nl,
   write('Time : '), write(AStarTime), write(' minutes'), nl,
   write('Cost : Rs.'), write(AStarCost), nl,

   write('========================================').
```

OUTPUT

DISPLAY ALL AVAILABLE ROUTES:



AFTER SELECTING THE SUITABLE PATH

